NAIC Teleconnection Demonstrator FAQ

Welcome to the comprehensive FAQ for the NAIC Teleconnection Demonstrator, an innovative Jupyter-based framework that integrates machine learning (ML) and high-performance computing (HPC) to analyze climate teleconnections, with a focus on Atlantic Multidecadal Variability (AMV) and Pacific Decadal Variability (PDV). This document is organized into five categories to guide climate scientists, data scientists, policy makers, and enthusiasts in understanding, setting up, and leveraging this powerful tool for advanced climate research. All information aligns with the project's technical documentation to ensure accuracy and consistency.

🌍 Category A: Understanding the Demonstrator

1. What is the NAIC Teleconnection Demonstrator and what problem does it solve?

The NAIC Teleconnection Demonstrator is a cutting-edge platform that combines machine learning and high-performance computing to uncover teleconnections—statistical relationships between climate indices like the Atlantic Multidecadal Oscillation (AMO) and North Atlantic Oscillation (NAO). Traditional climate models often struggle with the non-linear complexity of these relationships, which are critical for predicting long-term climate trends like AMV and PDV. By analyzing 65 climate indices across centuries of data, the demonstrator delivers precise, decadal-scale forecasts, empowering applications from disaster preparedness to sustainable policy-making.

2. What are climate teleconnections and why are they important?

Climate teleconnections are large-scale patterns linking climate events across distant regions, such as Pacific sea surface temperatures influencing Northern Hemisphere precipitation. They are vital for forecasting phenomena like El Niño, AMV-driven temperature shifts, or droughts, enabling proactive planning for agriculture, infrastructure, and climate resilience. Using historical and simulated data from the Norwegian Earth System Model (NorESM1-F), the demonstrator identifies these connections, providing actionable insights for long-term climate prediction.

3. Which climate indices are analyzed in this demonstrator?

The demonstrator processes 65 climate indices from NorESM1-F simulations, covering:

- **Global Surface Temperature**: `glTSann`, `nhTSann`, `shTSann` for global and hemispheric trends.
- **Sea Surface Temperature (SST)**: `amoSSTann`, `satlSSTann`, `ensoSSTjfm` for oceanic variability.
- **Sea Ice Concentration (SIC)**: `nhSICmar`, `nhSICsep`, `shSICmar`, `shSICsep` for polar ice dynamics.
- **Precipitation**: `nchinaPRjja`, `yrvPRjja`, `ismPRjja` for regional rainfall.
- **Pressure**: `naoPSLjfm`, `eapPSLjfm`, `scpPSLjfm` for atmospheric dynamics.
- **Ocean Circulation**: `AMOCann`, `traBO`, `traBS` for global currents.

A detailed list is available in the project's CLIMATE-INDICES.md file, supporting comprehensive AMV and PDV analysis.

4. What makes this demonstrator different from traditional climate modeling approaches?

Unlike traditional models reliant on linear regression, the NAIC Teleconnection Demonstrator uses advanced ML algorithms—Linear Regression, Random Forests, Multi-Layer Perceptrons (MLPs), and XGBoost—to capture complex, non-linear teleconnections. Coupled with HPC's parallel processing, it efficiently analyzes vast datasets, reducing computation time for millions of index combinations. This approach delivers superior accuracy and scalability, revolutionizing climate research for AMV, PDV, and beyond.

5. What are the main goals of the NAIC Teleconnection Demonstrator?

The demonstrator aims to:

- **Uncover Significant Teleconnections**: Identify relationships between indices like AMO and NAO to enhance AMV and PDV understanding.
- **Forecast Climate Indices**: Predict indices decades ahead for robust long-term modeling.
- **Optimize with HPC**: Leverage HPC to accelerate computations, enabling scalable analysis of complex climate data.

These goals position the demonstrator as a game-changer for climate prediction.

6. How does machine learning help in understanding climate variability?

Machine learning transforms climate variability analysis by:

- **Pattern Recognition**: Detecting subtle patterns in time-series data, such as lagged NAO-AMO relationships.
- **Non-Linear Modeling**: Capturing complex interactions critical for AMV and PDV.
- **Feature Importance**: Highlighting key predictors using models like Random Forests.

The demonstrator predicts indices up to decades ahead using rolling windows, with performance evaluated via correlation coefficients and Mean Absolute Error (MAE), ensuring accurate and interpretable results.

7. What datasets are used in the demonstrator?

The demonstrator uses NorESM1-F simulation data, including:

- **Historical SLOW Forcing (850–2005 AD)**: Low solar variability, volcanic activity, and anthropogenic effects.
- **Historical HIGH Forcing (850–2005 AD)**: High solar variability for comparative analysis.
- **Pre-Industrial Control (PICNTL)**: 1,000 years of constant 1850-level forcing.

Stored in the `dataset/` folder, these datasets cover 65 indices, providing a robust foundation for teleconnection analysis.

8. Can this demonstrator be applied to other regions or indices?

Yes, the demonstrator's modular design supports custom datasets for any region or index. Users can add time-series data to the `dataset/` folder, ensuring the format (time as first column, followed by indices) matches the NorESM1-F data. Minor script tweaks may be needed, enabling global applications beyond AMV and PDV, such as regional weather or novel indices.

9. What are some real-world applications of this analysis?

The demonstrator's insights enable:

- **Decadal Forecasting**: Predict AMV/PDV trends for long-term planning.
- **Agriculture**: Optimize crop strategies using precipitation forecasts.
- **Disaster Preparedness**: Anticipate extreme events like droughts linked to teleconnections.
- **Policy-Making**: Inform climate policies with reliable projections.

Sample results in the `results/` folder, like `amoSSTann` predictions, highlight validated teleconnections for practical impact.

10. Who can benefit from using this demonstrator?

The demonstrator serves:

- **Climate Scientists**: Exploring AMV/PDV teleconnections.
- **Data Scientists**: Applying ML to climate datasets.
- **Environmental Researchers**: Investigating interdisciplinary impacts.
- **Policy Makers**: Using forecasts for sustainable decisions.
- **Enthusiasts**: Experimenting with ML and HPC.

Its accessible design empowers diverse users to advance climate research.

🖥️ Category B: Infrastructure & Resources

1. How do I know what kind of resources I need for a demonstrator like this (parallel, CPU, GPU)?

Analyzing 65 climate indices with over 1,300 data points each generates millions of combinations, requiring HPC for efficiency. Parallel CPU processing is the primary approach, as the computational bottleneck is task volume, not data size. While GPUs can accelerate MLPs or XGBoost, CPUs are prioritized for interpretable models like Random Forests, ensuring robust teleconnection analysis on Sigma2's HPC clusters.

2. Where can I find the list of resources I could use?

The NAIC Teleconnection Demonstrator is optimized for Sigma2, Norway's HPC infrastructure, using SLURM for job management. Scripts in the `scripts/` folder (e.g., `.sbatch`, `.sh`) enable parallel job submission on clusters like Saga. Sigma2's documentation details available clusters, CPU/GPU configurations, and access procedures, ensuring users can select appropriate resources.

3. How much would it cost to run this on HPC?

Sigma2 allocates resources via quotas (CPU/GPU hours, storage). Researchers apply for quotas, often receiving thousands of compute hours. Parallel job execution, supported by the demonstrator's SLURM scripts, minimizes runtime for cost efficiency. For details, consult Sigma2's website for transparent resource planning guidelines.

4. What are the trade-offs between using CPUs and GPUs for ML in climate modeling?

- **GPUs**: Accelerate tensor-heavy models (MLPs, XGBoost) for speed.
- **CPUs**: Preferred for interpretable models (Random Forests, smaller MLPs) critical for understanding teleconnections.

The demonstrator prioritizes CPUs for transparency, ensuring results are scientifically meaningful across HPC environments.

5. How do I request access to specific hardware (e.g., GPUs) on SLURM?

Include in your SLURM script:

```
#SBATCH --gres=gpu:1
```

Verify GPU availability on clusters like Saga. CPUs are sufficient for most tasks, aligning with the demonstrator's design for accessibility.

6. What are the limitations of shared HPC environments?

Sigma2's shared HPC environment includes:

- **Queue Times**: High demand may delay jobs, mitigated by SLURM scheduling.
- **Fair-Share Policies**: Usage limits ensure equitable access.
- **Login Node Misuse**: Running tasks on login nodes disrupts systems; use compute nodes.

Sigma2's support teams assist with access or scheduling issues.

7. Can I run this demonstrator on a local machine or cloud instead of HPC?

- **Local Machines**: Suitable for small-scale testing but inadequate for full analyses.
- **Cloud Platforms**: Feasible but costly for large-scale runs.

HPC via Sigma2 is recommended for its scalability, as the demonstrator is optimized for SLURM-based environments.

8. What is the role of SLURM in managing HPC jobs?

SLURM manages:

- **Job Scheduling**: Prioritizes tasks for efficiency.
- **Resource Allocation**: Assigns CPUs, GPUs, and memory.
- **Monitoring**: Tracks job progress.

- **Fair-Share Enforcement**: Balances usage.

The demonstrator's SLURM scripts ensure seamless execution on Sigma2 clusters.

9. How do I monitor resource usage during job execution?

Use SLURM commands:

```
squeue -u $USER  # View active jobs
sacct            # Check job history
```

CSV logs in `results/` provide performance metrics, aiding resource optimization.

10. What happens if my job exceeds the allocated time or resources?

SLURM terminates jobs exceeding quotas, but the demonstrator's checkpointing allows resumption from the last completed step. Resubmit with adjusted settings using SLURM logs for guidance.

🧰 Category C: Prerequisites & Getting Started

1. What are the prerequisites to adopt this demonstrator for my work?

You need:

- **Python 3.8+**: For library compatibility.
- **Basic Python/ML Knowledge**: Familiarity with data handling and models like Random Forests.
- **Jupyter Notebooks**: For interactive analysis via `demonstrator.ipynb`.
- **SLURM Familiarity (Optional)**: Scripts simplify HPC usage.

These requirements ensure accessibility for diverse users.

2. How do I learn to use HPC for my work?

Leverage:

- Work Package 3 training from the Norwegian AI Cloud.
- Sigma2's documentation and online HPC/ML tutorials.
- NAIC team support for tailored guidance.

The demonstrator's scripts provide hands-on HPC experience.

3. What skills or tools do I need to start using this demonstrator?

- **Git**: For cloning (`git@gitlab.sigma2.no:naic/wp7/d7.2-Use-case1.git`) and version control.
- **Python/Jupyter**: For running scripts and notebooks.
- **Command-Line Basics**: For SLURM and HPC navigation.

Clear documentation ensures a smooth start.

4. How do I install the required dependencies?

Per the project's installation steps:

1. Purge modules: `module purge` .
2. Load Python: `module load Python/3.10.4-GCCcore-11.3.0` .
3. Create environment: `python -m venv ../env; source ../env/bin/activate` .
4. Install: `pip install -r requirements.txt` .

This installs `numpy` , `pandas` , `scikit-learn` , `xgboost` , etc.

5. Where can I learn more about using HPC for ML?

- Sigma2 documentation and GitHub wikis.
- Work Package 3 training programs.
- The demonstrator's scripts and `demonstrator.ipynb` for practical learning.

These resources empower users to master HPC-ML integration.

6. What datasets are required and where do I get them?

NorESM1-F data (SLOW, HIGH, PICNTL simulations) is included in the `dataset/` folder. Add custom datasets in the same format (time as first column, followed by indices) for flexibility.

7. How do I configure the environment for SLURM-based execution?

Use provided SLURM scripts:

```
module load Python/3.10.4-GCCcore-11.3.0
sbatch interactive-job-slurm.sh
```

These handle module loading and job submission for HPC.

8. What is the structure of the project repository?

- **dataset/**: NorESM1-F data and configurations.
- **scripts/**: Python and SLURM scripts.
- **results/**: CSV files and visualizations.
- **notebooks/**: Jupyter notebooks, with `demonstrator.ipynb` as the entry point.

9. How do I run the demonstrator for the first time?

1. Clone: `git clone git@gitlab.sigma2.no:naic/wp7/d7.2-Use-case1.git`.
2. Set up environment and install dependencies.
3. Open `demonstrator.ipynb` in Jupyter.
4. Run:

```
python scripts/lrbased_teleconnection/main.py --target_feature=amo3 --mode
```

10. What are common setup issues and how do I fix them?

- **SLURM Misconfiguration**: Check Sigma2 documentation for partition/account names.
- **Python Version**: Use Python 3.10.4.
- **Dependencies**: Re-run `pip install -r requirements.txt`.
- **Environment Variables**: Ensure `SLURM_SUBMIT_DIR`, `SLURM_JOB_ACCOUNT`, etc., are set.

⚙️ Category D: Technical Setup & Execution

1. How do I set up JupyterLab on an HPC resource?

Use `interactive-job-slurm.sh` to launch JupyterLab on a compute node, with port forwarding (e.g., `localhost:9648`) for browser access, ensuring efficient resource use.

2. How do I use GPUs to perform tasks in this demonstrator?

Request GPUs in SLURM:

```
#SBATCH --partition=gpu
```

The demonstrator auto-detects GPUs for XGBoost/MLPs, but CPUs are prioritized for interpretability.

3. What does the error “library not found” mean and how do I fix it?

Causes:

- Missing `requirements.txt` libraries.
- Unloaded modules.
- Version mismatches.

Fixes:

- Reinstall: `pip install -r requirements.txt`.
- Load: `module load Python/3.10.4-GCCcore-11.3.0`.
- Use provided SLURM scripts.

4. Why is my job taking too long or not completing?

Check `results/` CSV logs for progress. Increase parallel jobs or request more Sigma2 resources to handle combinatorial complexity.

5. How do I debug failed SLURM jobs?

Review `.out` and `.err` files for errors. Combine logs or consult Sigma2 support for cluster-specific issues.

6. What does “Parallel Evaluation” mean in this context?

Each model-index pair is processed independently across HPC nodes, accelerating analysis via SLURM’s parallelization for scalability.

7. How do I modify the lag or feature settings in the model?

Use:

```
python scripts/lrbased_teleconnection/main.py --step_lag=5
```

Adjusts lagged data for predictions (e.g., 5 years).

8. How do I visualize model performance?

Visualizations in `results/` include:

- Correlation plots (actual vs. predicted).
- MAE plots for accuracy.
- Feature importance plots for key predictors.

9. Can I integrate new ML models into the pipeline?

Yes, extend `scripts/lrbased_teleconnection/main.py` with scikit-learn/XGBoost models or custom preprocessing for flexibility.

10. How do I ensure reproducibility of my results?

- Set random seeds.
- Log configurations in `results/`.
- Use Git for version control.

Results are highly consistent despite minor stochasticity.

📊 Category E: Data Access, Sharing & Reproducibility

1. How do I get training data for the demonstrator?

NorESM1-F data (SLOW, HIGH, PICNTL) is in the `dataset/` folder, covering 65 indices.

2. Where do I upload my results?

Upload `results/` CSVs to Google Drive, institutional storage, or repositories like Zenodo.

3. How do I share my results with others?

Share `demonstrator.ipynb` with embedded descriptions, visualizations, and context for comprehensive communication.

4. Can I use the benchmarking data in my own research?

Yes, with attribution to the NAIC Teleconnection Demonstrator, providing valuable teleconnection insights.

5. How do I cite this demonstrator in a publication?

Cite as “NAIC Teleconnection Demonstrator” (DOI pending). Contact the NAIC team for guidance.

6. Can the demonstrator authors be co-authors on my paper?

Yes, if they contribute significantly. Contact the NAIC team to collaborate.

7. How do I anonymize data before sharing it?

Remove SLURM account names, Sigma2 references, or rename datasets, as the data is open and non-personal.

8. What’s the best way to collaborate on extending the demonstrator?

Use Git: create branches, open issues, submit pull requests for trackable collaboration.

9. How do I contribute improvements or bug fixes?

Fork the repository, make changes, and submit a pull request for NAIC team review.

10. What’s the roadmap for future updates to the demonstrator?

Future updates depend on community interest, potentially adding models or datasets to enhance AMV/PDV analysis.